

DNS.SetSolarWindsRecordToActiveServer.ps1

PowerShell script - no load balancer and no SNMP. It is told which SolarWinds server has become active and points the DNS record straight at it.

234 lines · Scripts/DNS.SetSolarWindsRecordToActiveServer.ps1

```
1 #Requires -Version 5.1
2
3 <#
4 .SYNOPSIS
5     Points the SolarWinds DNS record at the SolarWinds server that has just
6     become active. No load balancer, no web servers, no SNMP.
7
8 .DESCRIPTION
9     This is the Alteon-free variant of DNS.SetSolarWindsRecordToActiveLB.v2.ps1.
10    That script asks a Radware load balancer over SNMP how the web servers
11    behind it are doing, and picks a load balancer accordingly. This one does
12    not ask anything: it is told which SolarWinds server is now active, and
13    points the record straight at it.
14
15    Being told is the point. SolarWinds decided which server is active, so
16    SolarWinds is the authority on it. Anything the script worked out for
17    itself would at best reproduce that answer and at worst contradict it.
18
19    It is meant to run as an alert action on the server that has just become
20    active, with that server's address passed in -ActiveServer. Nothing is
21    written when the record already holds the wanted address, so it is also
22    safe to run on a schedule.
23
24 .PARAMETER ActiveServer
25     The address of the SolarWinds server that is now active, and the address
26     the record will be pointed at. It has to match either PrimaryDcSolarWindsIp
27     or SecondaryDcSolarWindsIp, otherwise the script fails instead of quietly
28     doing nothing.
29
30 .PARAMETER DnsServer
31     The DNS server to read the record from and write it back to. It has to hold
32     a writable copy of the zone. When it is left out the script walks the DNS
33     servers configured on this host and takes the first one that answers a ping.
34
35 .PARAMETER RecordTtlSeconds
36     When greater than zero, the TTL written onto the record. A failover only
37     takes effect once the old TTL has expired everywhere, so a low value (30-60)
38     is worth setting here.
39
40 .EXAMPLE
41     .\DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.10.1 -WhatIf
42
43     Shows what would change without touching DNS.
44
45 .EXAMPLE
46     .\DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.20.1
47
48 .NOTES
49     Exit codes: 0 done (changed or nothing to change), 1 failed.
50
51     Call it with -File, not -Command:
52
53         powershell.exe -NoProfile -File DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.10.1
54
55     A parameter that fails validation stops the script before it runs, and only
56     -File turns that into exit code 1. Under -Command the caller reads a stale
57     $LASTEXITCODE instead, so a typo in the arguments looks like a clean run.
58
```

```

59 The record points at the SolarWinds server itself. If a load balancer sits
60 in front of SolarWinds in your environment, this is the wrong script for it:
61 use DNS.SetSolarWindsRecordToActiveLB.v2.ps1, which picks between the two
62 load balancers on the health of the web servers behind them.
63 #>
64
65 [CmdletBinding(SupportsShouldProcess)]
66 param(
67     [Parameter(Mandatory, Position = 0)]
68     [ValidateNotNullOrEmpty()]
69     [string] $ActiveServer,
70
71     [ValidateNotNullOrEmpty()]
72     [string] $RecordName = 'OurSolar',
73
74     [ValidateNotNullOrEmpty()]
75     [string] $ZoneName = 'OurZone',
76
77     [string] $DnsServer,
78
79     [ValidateNotNullOrEmpty()]
80     [string] $PrimaryDcSolarWindsIp = '10.10.10.1',
81
82     [ValidateNotNullOrEmpty()]
83     [string] $SecondaryDcSolarWindsIp = '10.10.20.1',
84
85     [ValidateRange(0, 86400)]
86     [int] $RecordTtlSeconds = 0,
87
88     [string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\dns-active-server.log')
89 )
90
91 Set-StrictMode -Version Latest
92 $ErrorActionPreference = 'Stop'
93
94 function Write-Log {
95     param(
96         [Parameter(Mandatory)]
97         [AllowEmptyString()]
98         [string] $Message,
99
100         [ValidateSet('INFO', 'WARN', 'ERROR')]
101         [string] $Level = 'INFO'
102     )
103
104     $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
105     Write-Information -MessageData $line -InformationAction Continue
106
107     # -WhatIf:$false so that a -WhatIf run still leaves a trace of what it
108     # decided; the log is a record, never one of the changes being previewed.
109     try {
110         $directory = Split-Path -Path $script:LogPath -Parent
111         if ($directory -and -not (Test-Path -LiteralPath $directory)) {
112             New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false | Out-Null
113         }
114         Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false
115     }
116     catch {
117         Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
118     }
119 }
120
121 function Resolve-DnsServerAddress {
122     # Walks the DNS servers configured on this host and returns the first one
123     # that answers a ping. Only adapters with IP enabled are looked at, so

```

```

124 # disconnected or disabled NICs cannot contribute a stale address.
125 [OutputType([string])]
126 param()
127
128 $adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter 'IPEnabled = True'
129
130 foreach ($adapter in $adapters) {
131     foreach ($address in $adapter.DNSServerSearchOrder) {
132         if (Test-Connection -ComputerName $address -Count 1 -Quiet -ErrorAction SilentlyContinue) {
133             Write-Log "Using DNS server $address, configured on '$($adapter.Description)'."
134             return [string]$address
135         }
136
137         Write-Log "DNS server $address did not answer a ping, trying the next one." -Level WARN
138     }
139 }
140
141 throw 'None of the DNS servers configured on this host answered a ping. Pass -DnsServer to name one explicitly.'
142 }
143
144 function Get-SolarWindsRecord {
145     param(
146         [Parameter(Mandatory)][string] $Server
147     )
148
149     $records = @(Get-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -Name $RecordName -
150 RRTYPE A)
151
152     if ($records.Count -eq 0) {
153         throw "Zone '$ZoneName' on $Server holds no A record named '$RecordName'."
154     }
155
156     if ($records.Count -gt 1) {
157         throw "Zone '$ZoneName' on $Server holds $($records.Count) A records named '$RecordName'. This script
158 handles a single A record, so the duplicates have to be resolved first."
159     }
160
161     return $records[0]
162 }
163
164 function Set-SolarWindsRecord {
165     # Set-DnsServerResourceRecord needs the untouched record and the edited copy
166     # as two separate objects, hence the Clone.
167     param(
168         [Parameter(Mandatory)][string] $Server,
169         [Parameter(Mandatory)][string] $TargetIp
170     )
171
172     $current = Get-SolarWindsRecord -Server $Server
173     $updated = $current.Clone()
174     $updated.RecordData.IPv4Address = [System.Net.IPAddress]::Parse($TargetIp)
175
176     if ($RecordTtlSeconds -gt 0) {
177         $updated.TimeToLive = [System.TimeSpan]::FromSeconds($RecordTtlSeconds)
178     }
179
180     Set-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -OldInputObject $current -
181 NewInputObject $updated
182
183     $after = Get-SolarWindsRecord -Server $Server
184     $afterIp = $after.RecordData.IPv4Address.IPAddressToString
185
186     if ($afterIp -ne $TargetIp) {
187         throw "The record was written but still reads $afterIp instead of $TargetIp."
188     }
189 }

```

```

185     }
186 }
187
188 try {
189     Write-Log "==== Run started for active server $ActiveServer ====="
190
191     # The address is checked against the two servers the script knows rather
192     # than used as given, so a typo in the alert action cannot point the record
193     # at something that was never a SolarWinds server.
194     if ($ActiveServer -eq $PrimaryDcSolarWindsIp) {
195         $activeName = 'PrimaryDC'
196     }
197     elseif ($ActiveServer -eq $SecondaryDcSolarWindsIp) {
198         $activeName = 'SecondaryDC'
199     }
200     else {
201         throw "ActiveServer '$ActiveServer' is neither the PrimaryDC SolarWinds server
202         ($PrimaryDcSolarWindsIp) nor the SecondaryDC one ($SecondaryDcSolarWindsIp)."
203     }
204
205     Write-Log "$activeName is active, so the record should point at $ActiveServer."
206
207     $dnsServerAddress = if ($DnsServer) { $DnsServer } else { Resolve-DnsServerAddress }
208
209     $record = Get-SolarWindsRecord -Server $dnsServerAddress
210     $currentIp = $record.RecordData.IPv4Address.IPAddressToString
211     Write-Log "$RecordName.$ZoneName currently points at $currentIp with a TTL of $($record.TimeToLive)."
212
213     if ($currentIp -eq $ActiveServer) {
214         Write-Log "Nothing to do, the record already points at the $activeName server ($ActiveServer)."
215         exit 0
216     }
217
218     $target = "$RecordName.$ZoneName on $dnsServerAddress"
219     $action = "Repoint from $currentIp to $ActiveServer, the $activeName server"
220
221     if (-not $PSCmdlet.ShouldProcess($target, $action)) {
222         Write-Log "WhatIf: would have repointed $RecordName.$ZoneName from $currentIp to $ActiveServer
223         ($activeName server)."
224         exit 0
225     }
226
227     Set-SolarWindsRecord -Server $dnsServerAddress -TargetIp $ActiveServer
228     Write-Log "Repointed $RecordName.$ZoneName from $currentIp to $ActiveServer, the $activeName server."
229     exit 0
230 }
231 catch {
232     Write-Log "Run failed: $($_.Exception.Message)" -Level ERROR
233     Write-Log $_.ScriptStackTrace -Level ERROR
234     Write-Error -Message $_.Exception.Message -ErrorAction Continue
235     exit 1
236 }

```